

Vue.js 반응형 레이아웃 & 차트 시각화 등

실무 프로젝트로 배우는 모던 프론트엔드 대시보드 개발



학습 목표

반응형 레이아웃 설계 (CSS Grid, Bootstrap)

Chart.js + vue-chartjs 데이터 시각화

Vuetify 활용 다크모드 & 전문 UI



기술 스택

Vue 3

Vite

CSS Grid

Bootstrap

Chart.js

Vuetify



최종 결과물

실습 기반의 금융 대시보드 UI를 완성합니다.



전체 프로젝트 코드 제공

Table of Contents

01

반응형 레이아웃 & 그리드 시스템

CSS GRID

BOOTSTRAP

02

차트 시각화

CHART.JS

VUE-CHARTJS

03

Vuetify UI & 다크모드

VUETIFY 3

THEMING

04

전체 프로젝트 코드

통합 소스 코드 리뷰

05

마무리 & 다음 단계

확장 가이드

CHAPTER

01

반응형 레이아웃 설계와 그리드 시스템

CSS Grid와 Bootstrap을 활용하여
다양한 화면 크기에 대응하는
유연한 UI 구조를 설계합니다.



학습 목표

Learning Objectives

반응형 레이아웃 개념 이해

유동적인 너비와 브레이크포인트(Breakpoint)의 원리

CSS Grid로 카드형 레이아웃 만들기

display: grid와 미디어쿼리 활용 실습

Bootstrap Grid 시스템 활용

클래스 기반으로 빠르게 반응형 화면 구성하기

실제 대시보드 만들 때 가장 먼저 부딪히는 게 레이아웃입니다.
오늘은 두 가지 핵심 기술로 직접 카드형 UI를 만들어봅니다.

왜 반응형 레이아웃이 필요한가?

사용자 환경의 다양성

단일 해상도 기준의 개발은 불가능합니다.

사용자는 다양한 디바이스로 금융 서비스를 접속합니다.

Mobile

360px ~

Tablet

768px ~

Laptop

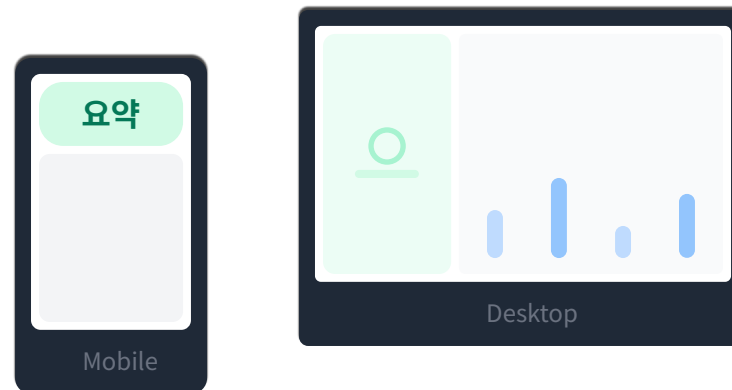
1024px ~

Wide

1920px ~

핀테크 서비스 사례

USE CASE



모바일: 핵심 지표 요약 PC: 차트, 테이블, 필터 등 복합 UI

One Code, Any Screen

코드 한 번 작성으로
모든 해상도를 커버하는 효율성

반응형 레이아웃의 핵심 요소

1. 유동적인 너비 (Fluid Width)

고정 픽셀(px) 대신 화면 크기에 따라 변하는 상대 단위를 사용합니다.

% (Percentage)

fr (Fraction)

vw / vh

66%

2. 브레이크포인트 (Breakpoint)

레이아웃이 변경되는 기준 너비 지점입니다. 디바이스 종류에 대응합니다.

600px

1024px

3. 미디어쿼리 (Media Query)

CSS에서 화면 조건에 따라 다른 스타일을 적용하는 기술입니다.

```
@media (max-width: 600px) {  
  .container {  
    flex-direction: column;  
  }  
}
```

4. 레이아웃 패턴

화면이 좁아질 때 배치가 어떻게 변할지 결정합니다.



Desktop



Mobile

CSS Grid 소개 및 기본 문법

2차원 레이아웃

Grid는 행(Row)과 열(Column)을 동시에 제어하는 2차원 시스템입니다.

핵심 속성

display: grid
grid-template-columns
gap

오늘의 목표

카드 8개를 그리드로 배치하고, 화면 크기에 따라 4열 → 2열
→ 1열로 자동 변경되는 반응형 구조 만들기

```
style.css

1 .grid-container {
2   /* 이 컨테이너를 그리드 레이아웃으로 선언 */
3   display: grid;
4   /* 가로 공간을 4등분 (1fr * 4) */
5   grid-template-columns: repeat(4, 1fr);
6   /* 행(Row)의 높이 설정 (선택사항) */
7   grid-template-rows: auto;
8   /* 아이템 사이의 간격 16px */
9   gap: 16px;
10  padding: 20px;
11 }
```

fr Fraction (비율 단위)

repeat() 반복 함수

미디어쿼리로 반응형 만들기

브레이크포인트

화면 너비(Width)가 특정 지점에 도달했을 때 레이아웃을 변경하는 기준점입니다.

반응형 규칙

PC	기본값	4열
Tablet	$\leq 1024\text{px}$	2열
Mobile	$\leq 600\text{px}$	1열

Key Point

미디어쿼리 안에서 변경할 속성만 덮어쓰기(Override) 하면 됩니다. 나머지는 그대로 상속됩니다.

style.css

```
1 /* 기본 PC 화면 (1024px 초과) */
2 .grid-container {
3   grid-template-columns :repeat(4,1fr);
4 }
5
6 /* 태블릿 (1024px 이하) → 2열로 변경 */
7 @media (max-width :1024px ){
8   .grid-container {
9     grid-template-columns :repeat(2,1fr);
10  }
11 }
12
13 /* 모바일 (600px 이하) → 1열로 변경 */
14 @media (max-width :600px ){
15   .grid-container {
16     grid-template-columns :repeat(1,1fr);
17  }
```

ResponsiveGrid.vue 전체 코드

Vue의 역할

템플릿은 단순히 구조만 표현합니다. 복잡한 로직 없이 v-for로 카드만 생성하고, 배치는 전적으로 CSS에 위임합니다.

CSS의 역할

Grid 선언: 카드들을 2차원 그리드로 배치

미디어쿼리: 화면 폭에 따라 열 개수 자동 조절
(4열 → 2열 → 1열)

src/components/ResponsiveGrid.vue

```
1 <template>
2 <divclass="grid-container" >
3 <divclass="card"v-for="n in 8":key="n">
4 CSS Grid 카드 {{n}}
5 </div>
6 </div>
7 </template>

8 <style>
9 .grid-container {
10 display:grid;
11 grid-template-columns :repeat(4,1fr); /* 기본 4열 */
12 gap:16px;
13 padding:20px;
14 }

15 /* 태블릿: 1024px 이하에서 2열로 변경 */
16 @media(max-width:1024px){
17 .grid-container {
```


Bootstrap 프레임워크 개요



Bootstrap이란?

The world's most popular front-end open source toolkit

트위터(Twitter)에서 개발한 오픈소스 UI 프레임워크로, **HTML, CSS, JS**를 기반으로 반응형 웹을 빠르고 쉽게 제작할 수 있도록 도와주는 도구입니다.



역사 (History)

2011년 Mark Otto와 Jacob Thornton이 개발. 현재 v5 버전까지 진화하며 웹 표준을 선도.



컴포넌트 (Components)

버튼, 카드, 네비게이션, 모달 등 미리 디자인된 수많은 UI 요소 제공.



빠른 개발 속도

미리 정의된 클래스로
즉각적인 UI 구축



Mobile First

강력한 12컬럼
반응형 그리드 시스템

장점과 단점

VS

장점 (Pros)

- ✓ 높은 브라우저 호환성
- ✓ 방대한 커뮤니티 & 문서
- ✓ 일관된 디자인 품질

단점 (Cons)

- ✗ 정형화된 디자인 (Bootstrap Look)
- ✗ 불필요한 스타일 로딩 가능성
- ✗ DOM 구조가 복잡해질 수 있음

Bootstrap Breakpoints & Sizes

📱 그리드 시스템 브레이크포인트 (v5 기준)			
min-width 기준			
Prefix	Size Info	Dimensions	Device Example
xs	X-Small	< 576px	📱 모바일 세로 (iPhone, Galaxy)
sm	Small	≥ 576px	📱 모바일 가로 (Large Phones)
md	Medium	≥ 768px	📱 태블릿 (iPad Mini, Galaxy Tab)
lg	Large	≥ 992px	💻 소형 노트북 / 태블릿 가로
xl	X-Large	≥ 1200px	💻 일반 데스크탑 모니터
xxl	XX-Large	≥ 1400px	💻 고해상도 대형 모니터

</> 클래스 사용 패턴

```
<div class="col-12 col-md-6 col-xl-4">  
  
</div>
```

✓ col-12: 기본(모바일)에서 전체 너비 (1열)

✓ col-md-6: 768px 이상에서 절반 너비 (2열)

✓ col-xl-4: 1200px 이상에서 1/3 너비 (3열)

xs

sm

md

lg

xl

Bootstrap Grid 한눈에 보기

1 INSTALLATION

```
→ project npm install bootstrap
# main.js에서 import 'bootstrap/dist/css/bootstrap.min.css'
import 'bootstrap/dist/js/bootstrap.bundle.min.js'
```

2 USAGE STRUCTURE

```
1<!-- 전체 폭 중앙 정렬 -->
2<div class="container">
3
4<!-- 가로줄 (Row) -->
5<div class="row">
6
7<!-- 12칸 중 4칸 차지 (33%) -->
8<div class="col-4">
9<p>Column 1</p>
10</div>
```

클래스 기반 레이아웃

Class-based System

CSS를 직접 작성하지 않고, HTML 클래스 이름만으로 레이아웃을 제어합니다. 생산성이 매우 높고 팀 프로젝트에서 일관성을 유지하기 쉽습니다.

`.container`

`.row`

`.col-*`

핵심 포인트

12 컬럼 시스템

화면 가로폭을 12등분하여 자유롭게 조합 (3+3+6, 4+4+4 등)

강력한 반응형

col-md-6처럼 브레이크포인트 접두사를 사용해 기기별 레이아웃 지정

CSS Grid vs Bootstrap Grid

CSS Grid



완전한 레이아웃 자유도



2차원 레이아웃 강력함



복잡한 디자인 구현



학습 곡선 존재



직접 CSS 작성 필요

Bootstrap Grid



빠른 개발 속도



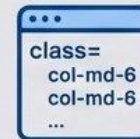
팀 협업 용이



일관된 디자인 시스템



12칸 그리드 제한



클래스 의존적

CSS Grid vs Bootstrap Grid



CSS Grid

Custom Layout Design

이럴 때 선택하세요:

디자이너의 시안이 복잡하고 독창적일 때

2차원(가로+세로) 배치를 정밀하게 제어해야 할 때

HTML 구조를 단순하게 유지하고 싶을 때 (시멘틱 마크업)

Learning Curve



Bootstrap Grid

Rapid Prototyping

이럴 때 선택하세요:

빠르게 관리자 페이지나 대시보드를 만들 때

여러 개발자가 공통된 규칙으로 협업해야 할 때

12컬럼 기반의 표준 레이아웃으로 충분할 때

Learning Curve

